Localización del ligamento cruzado anterior en resonancias magnéticas de rodilla

Entregable 2: Preparación de los datos (CRISP-ML(Q), fase 2)

Christopher Zúñiga, C28730

Escuela de Ciencias de la Computación e Informática

Universidad de Costa Rica — CI-0148 Aprendizaje Automático

Octubre de 2026

Resumen

Este documento corresponde al segundo entregable del curso CI-0148 y desarrolla la fase 2 de CRISP-ML(Q): preparación de los datos. A partir del planteamiento de la fase 1 —localizar el ligamento cruzado anterior (LCA) en volúmenes de resonancia magnética (RM) del conjunto KneeMRI—, se construyó un pipeline reproducible que (i) inventaria y verifica los 917 volúmenes del conjunto, (ii) aplica una puerta de calidad automatizada que detiene el proceso ante cualquier inconsistencia, (iii) particiona los datos en un conjunto de prueba reservado y cinco pliegues de validación cruzada, agrupando por examen y estratificando por diagnóstico, antes de cualquier transformación, (iv) normaliza la intensidad por volumen y uniformiza el tamaño en plano sin alterar las coordenadas de la caja objetivo, y (v) genera un índice de 28 359 cortes para un modelo de aprendizaje profundo 2.5D. Se documenta además la estrategia para los dos desbalances del problema (cortes con y sin LCA, y severidad del diagnóstico). Todas las verificaciones de calidad pasaron: ningún examen aparece en más de una partición y la proporción de clases de cada partición se desvía como máximo 0.54 puntos porcentuales del total. El pipeline se acompaña de 31 pruebas unitarias y de documentación para reconstruir el conjunto. El documento cierra con una revisión honesta del plan: el entregable llega a tiempo según el calendario vigente del curso, pero el proyecto va unos diez días atrás respecto a su propio plan.

Palabras clave: preparación de datos, fuga de información, validación cruzada agrupada, desbalance de clases, resonancia magnética, CRISP-ML(Q).

Índice

1. Introducción **2**

2. Materiales y métodos **3**
2.1. Recursos utilizados . . . 3
2.2. Arquitectura del pipeline . . . 3
2.3. Manifiesto: inventario verificado de los datos . . . 4
2.4. Puerta de calidad . . . 5
2.5. Partición sin fuga de información . . . 6
2.6. Representación y transformaciones . . . 7
2.7. Índice de cortes 2.5D . . . 8
2.8. Estrategia para el desbalance . . . 9
2.9. Reproducibilidad . . . 9
2.10. Pruebas automatizadas y documentación . . . 10

3. Discusión de resultados **11**
3.1. Integridad y calidad de los datos . . . 11
3.2. Partición . . . 15
3.3. Índice de cortes y desbalance . . . 15
3.4. Seguimiento de la tabla de riesgos de la fase 1 . . . 16
3.5. Limitaciones . . . 16

4. Avance y revisión del plan **17**

5. Conclusiones **18**

1. Introducción

En la fase 1 de este proyecto se planteó una tarea de aprendizaje automático orientada a apoyar la lectura clínica de resonancias magnéticas de rodilla: **localizar automáticamente el ligamento cruzado anterior (LCA)** dentro del volumen, es decir, predecir la caja delimitadora tridimensional que lo contiene. El LCA es una banda delgada y oblicua que aparece en apenas unos pocos cortes sagitales consecutivos; un especialista debe encontrarla antes de poder evaluar si está sana, parcialmente lesionada o rota. Automatizar ese primer paso de ubicación permite que el radiólogo, o un modelo posterior de clasificación, concentre su atención en la región correcta.

El conjunto de datos es KneeMRI [1, 2]: 917 resonancias de rodilla vistas de lado (cortes sagitales), con una configuración que oscurece la grasa para que ligamentos y líquido resalten mejor, adquiridas en un único equipo Siemens Avanto 1.5 T del Clinical Hospital Centre Rijeka (Croacia) entre 2006 y 2014. Cada examen trae un diagnóstico de severidad (`aclDiagnosis`: 0 sano, 1 parcial, 2 roto) y una caja delimitadora 3D alrededor del LCA trazada manualmente por los autores. En la fase 1 se estableció como línea base una caja fija promedio, que alcanza un IoU medio de 0.391, y se fijó como criterio de éxito un IoU medio de al menos 0.5 en el conjunto de prueba.

Contexto de la preparación de datos. En CRISP-ML(Q) [3], la preparación de datos es la fase que transforma los datos crudos en un conjunto listo para entrenar, y suele ser la que más tiempo consume. El modelo de proceso exige, además, que cada tarea tenga un método de aseguramiento de calidad (Q) con un criterio verificable. Para este proyecto, el contexto impone tres riesgos concretos:

1. **Fuga de información** [4]. Algunos exámenes tienen varias series de la misma rodilla; si una serie cae en entrenamiento y otra en prueba, el modelo se evalúa sobre una rodilla que ya vio. Del mismo modo, cualquier estadística calculada con todos los datos antes de particionar filtra información del conjunto de prueba al entrenamiento.

2. **Desbalance**. El diagnóstico está desbalanceado (75.2 % sanos, 18.8 % parciales, 6.0 % rotos), y, como el LCA ocupa solo de 2 a 6 de los 21 a 45 cortes de cada volumen, alrededor de nueve de cada diez cortes no contienen el ligamento.

3. **Transformaciones irreproducibles**. En imágenes médicas abundan decisiones (tamaño, normalización, orientación) que, si se aplican de forma manual o en cuadernos ejecutados en desorden, no pueden reconstruirse ni auditarse.

Entre la fase 1 y esta fase se tomó además una decisión que condiciona todo el preprocesamiento: el modelado se hará con **aprendizaje profundo sobre una representación 2.5D** (cada corte se acompaña de sus dos vecinos), en lugar de un modelo clásico con características diseñadas a mano o de una red 3D sobre el volumen completo. La justificación se presenta en la Sección 2.6.

Organización del documento. La Sección 2 (Materiales y métodos) lista los recursos y herramientas utilizados y describe, con su justificación y muestras de código, cada etapa del pipeline: inventario, puerta de calidad, partición, transformaciones, índice de cortes, estrategia de desbalance y reproducibilidad. La Sección 3 analiza los resultados obtenidos y los contrasta con los criterios de calidad definidos en la fase 1. La Sección 4 revisa el avance frente al plan de la propuesta, y la Sección 5 presenta las conclusiones.

2. Materiales y métodos

2.1. Recursos utilizados

Datos. Se usó la versión completa de KneeMRI publicada en Zenodo [2] (917 archivos `.pck` con arreglos NumPy `uint16`, 6.2 GB, licencia CC BY-NC-ND 4.0) junto con el archivo `metadata.csv` de la redistribución de Kaggle [5]. La razón de usar dos fuentes se explica en la Sección 2.3.

Poder computacional. Todo el trabajo se ejecutó en una computadora portátil personal: procesador Apple M1 (8 núcleos), 16 GB de memoria unificada y macOS 26.6. El pipeline completo no requiere GPU y se ejecuta en unos 67 s, de los cuales 51 s corresponden al procesamiento de los volúmenes.

Software. Python 3.14.4 en un entorno virtual con dependencias fijadas en `requirements.txt`: NumPy 2.5.2 [6], pandas 3.0.5 [7], scikit-learn 1.9.1 [8], SciPy 1.18.1, PyYAML 6.0.3, Matplotlib 3.11.1 y pytest 9.1.1.

Recursos humanos. Un único estudiante. No se contó con un especialista clínico para revisar las anotaciones; esta limitación ya se había declarado en la fase 1 y se retoma en la Sección 3. Como apoyo se usó un asistente de programación basado en IA para revisar código y discutir decisiones de diseño; las decisiones y la implementación son del autor.

Referentes metodológicos. La organización del proyecto sigue la convención de Cookiecutter Data Science [9] (datos crudos inmutables, datos intermedios y procesados regenerables). Las decisiones de normalización siguen las de nnU-Net [10], la referencia de facto para el preprocesamiento de imágenes médicas, y la representación por cortes sigue la práctica de MRNet [11], el antecedente más citado de aprendizaje profundo en RM de rodilla.

2.2. Arquitectura del pipeline

La Figura 1 resume el flujo. El pipeline se rige por cuatro principios:

1. **Los datos crudos no se modifican nunca**. Todo lo que se genera vive en `datasets/interim/` y `datasets/processed/`, y puede borrarse y regenerarse.

2. **La lógica vive en funciones puras** dentro de `src/data/`: reciben datos y devuelven datos, sin leer configuración ni escribir archivos. Esto las hace probables de forma aislada y reutilizables en la fase 3.

3. **Un único punto de entrada**, `scripts/build_dataset.py`, lee la configuración, ejecuta las etapas en orden y escribe los resultados. Los cuadernos de Jupyter se usan solo para exploración y nunca producen artefactos de los que dependa el pipeline.

4. **Toda decisión está en la configuración** (`configs/prep.yaml`, Código 1), incluida la semilla aleatoria; ningún parámetro está escrito directamente en el código.

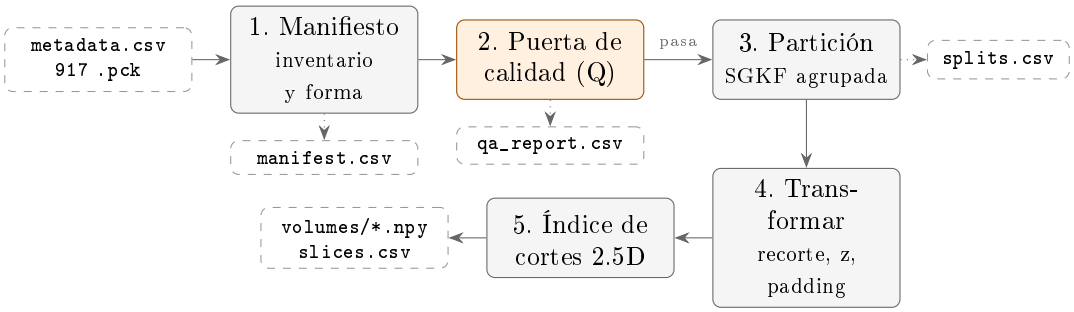


Figura 1: Pipeline de preparación de datos. Cada etapa es una función pura en `src/data/`; `scripts/build_dataset.py` las encadena y guarda los artefactos (recuadros punteados). Si la puerta de calidad encuentra cualquier problema, el pipeline se detiene antes de particionar.

```yaml
seed: 42
data:
  metadata_csv: datasets/raw/kneemridataset/metadata.csv
  volumes_dir: datasets/raw/volumetric_data
  interim_dir: datasets/interim
  processed_dir: datasets/processed
preprocess:
  clip_low_pct: 0.5
  clip_high_pct: 99.5
  pad_height: 320
  pad_width: 320
  storage_dtype: float16
split:
  n_test_splits: 6
  n_cv_folds: 5
  max_prop_diff: 0.02
```

Código 1: Configuración completa del pipeline (`configs/prep.yaml`).

2.3. Manifiesto: inventario verificado de los datos

El primer paso construye un *manifiesto*: una tabla con una fila por volumen que reúne los metadatos del conjunto con información verificada directamente sobre los archivos (ruta, existencia, forma, tipo de dato e intensidades mínima y máxima). Todas las etapas posteriores

leen esta tabla en lugar de recorrer carpetas, de modo que no puedan discrepar sobre qué volúmenes existen.

Datos faltantes. Al construir el primer manifiesto sobre la descarga de Kaggle se confirmó el hallazgo de la fase 1: solo 736 de los 917 archivos existían (faltaban las carpetas `vol09` y `vol10`), y entre los faltantes había 10 de los 55 casos de rotura completa. Como perder el 18 % de la clase más escasa comprometía la estratificación, se descargó el conjunto completo desde Zenodo [2]. Antes de usarlo se verificó que (i) los 917 nombres de `metadata.csv` tienen archivo correspondiente, sin archivos sobrantes, y (ii) los 736 archivos comunes a ambas fuentes son idénticos byte a byte (mismo hash MD5). Esto permite combinar el `metadata.csv` de Kaggle con los volúmenes de Zenodo sin ambigüedad.

Clave primaria. `examId` no es único: 7 exámenes tienen más de una serie (seis con dos y uno con tres), de modo que los 917 volúmenes corresponden a 909 exámenes. La clave es el par (`examId`, `seriesNo`), y el manifiesto falla de inmediato si encuentra un par duplicado, antes de abrir ningún archivo. Las rutas se guardan relativas a la carpeta de volúmenes, de modo que el manifiesto es portable entre máquinas.

2.4. Puerta de calidad

Antes de particionar, la función `run_qa_checks` evalúa el manifiesto con reglas fijas (Tabla 1). En lugar de detenerse en el primer error, recoge todos los problemas en un reporte (`qa_report.csv`) que se guarda siempre; si el reporte no está vacío, el pipeline se detiene. Un reporte vacío, con solo encabezados, es la evidencia de que la verificación se ejecutó y pasó. Como las reglas no aprenden nada de los datos, ejecutarlas sobre el conjunto completo no introduce fuga.

Cuadro 1: Verificaciones de la puerta de calidad y su justificación.

Verificación	Regla	Por qué importa
Etiquetas válidas	`aclDiagnosis` $\in \{0,1,2\}$, `kneeLR` $\in \{0,1\}$	Un valor inválido crearía una cuarta "clase" en la estratificación.
Archivo presente	existe el `.pck` de cada fila	Un volumen faltante cambiaría la partición en silencio.
Tamaño de la caja	ancho, alto y profundidad > 0	Una caja vacía no es un objetivo aprendible.
Caja dentro del volumen	en cada eje: inicio ≥ 0 e inicio + tamaño $\leq$ dimensión	La caja es la variable objetivo; si sale del volumen no puede aprenderse ni recortarse.
Tipo de dato	`uint16` en todos los volúmenes	La normalización asume enteros de 12 bits.
Intensidad	máximo > 0 y ≤ 4095	Detecta volúmenes vacíos o fuera del rango de 12 bits.

La verificación de límites no puede detectar si los ejes x e y de la caja están intercambiados,

porque casi todos los volúmenes miden 320×320. Por eso se complementó con una inspección visual, cuyos resultados se discuten en la Sección 3.

2.5. Partición sin fuga de información

La partición se hace **antes de cualquier transformación** y debe cumplir dos reglas a la vez:

- **Agrupar por** `examId`: todas las series de un mismo examen quedan en la misma partición.
- **Estratificar por** `aclDiagnosis`: cada partición conserva aproximadamente las proporciones 75/19/6. Con solo 55 casos de rotura, una partición aleatoria puede dejar al conjunto de prueba con muy pocos.

Las herramientas habituales de scikit-learn cumplen solo una de las dos reglas. Se compararon sobre el manifiesto real (Tabla 2): `train_test_split` estratificado separó un examen entre entrenamiento y prueba, y `GroupShuffleSplit` dejó entre 7 y 15 casos de rotura en prueba según la semilla. Solo `StratifiedGroupKFold` (SGKF) cumple ambas.

Cuadro 2: Comparación de estrategias de partición sobre los 917 volúmenes (prueba $\approx 1/6$ de los datos).

Método	Estratifica	Agrupa	Resultado observado
`train_test_split(stratify=y)`	Sí	No	1 examen repartido entre entrenamiento y prueba
`GroupShuffleSplit`	No	Sí	15, 9 o 7 roturas en prueba (semillas 0, 1, 2)
`StratifiedGroupKFold`	Sí	Sí	0 exámenes compartidos; 115/29/9 en prueba

SGKF no tiene un parámetro de tamaño de prueba: divide los datos en k pliegues. Por eso el conjunto de prueba se obtiene pidiendo $k = 6$ pliegues y reservando uno ($\approx$16.7 %); el resto se divide de nuevo con SGKF en 5 pliegues para la validación cruzada de la fase 3. Se eligió $k = 6$ frente a $k = 7$ porque deja 9 casos de rotura en prueba en lugar de 7, a cambio de solo 22 volúmenes menos para desarrollo. Este es el número que más importa, porque la rotura es la clase más difícil de localizar y la de mayor relevancia clínica. El Código 2 muestra el núcleo de la implementación.

```
df = manifest.copy().reset_index(drop=True)
y, groups = df['aclDiagnosis'], df['examId']

# Test set: one SGKF fold out of n_test_splits
test_splitter = StratifiedGroupKFold(n_splits=n_test_splits, shuffle=True,
    random_state=seed)
dev_idx, test_idx = next(test_splitter.split(df, y, groups=groups))
df['split'] = 'unassigned'
df.loc[test_idx, 'split'] = 'test'

```

```
# CV folds on the remaining development data
dev = df.iloc[dev_idx]
cv = StratifiedGroupKFold(n_splits=n_cv_folds, shuffle=True, random_state=seed)
for fold, (_, val_local) in enumerate(cv.split(dev, dev['aclDiagnosis'], groups=dev['
    examId'])):
    df.loc[dev.iloc[val_local].index, 'split'] = f'fold_{fold}'

verify_splits(df, n_cv_folds, max_prop_diff)
```

Código 2: Partición en dos niveles (`src/data/split.py`, abreviado).

La función `verify_splits` implementa como código tres criterios Q de la fase 1: (a) ningún `examId` aparece en más de una partición; (b) la proporción de cada clase en cada partición está a no más de 2 puntos porcentuales de la proporción total; y (c) cada volumen queda asignado exactamente una vez. Si cualquiera falla, el pipeline se detiene. Se comprobó deliberadamente que el criterio (b) puede fallar, bajando temporalmente la tolerancia a 0.001. El resultado se congela en `splits.csv`; las fases siguientes leen ese archivo y nunca vuelven a particionar.

2.6. Representación y transformaciones

Elección de la representación 2.5D. Se evaluaron tres opciones: un modelo clásico de scikit-learn con características diseñadas a mano, una red 3D sobre el volumen completo y una red 2D sobre cortes. Se eligió aprendizaje profundo con entrada **2.5D**: cada corte s se presenta junto con $s-1$ y $s+1$ como los tres canales de una imagen. Las razones son:

- Con 917 volúmenes, las redes preentrenadas en ImageNet (p. ej. ResNet [12]) son la herramienta más sólida, y esperan exactamente tres canales.
- El trabajo por cortes multiplica las muestras (28 359 cortes frente a 917 volúmenes), y es la práctica dominante en la literatura de RM de rodilla [11].
- La alternativa 3D exigiría remuestrear la profundidad a un número fijo de cortes. Con una caja de solo 2 a 6 cortes, comprimir 45 cortes en 32 deformaría precisamente el objeto que se quiere localizar. Este riesgo ya se había anticipado como contingencia en la tabla de riesgos de la fase 1.

La salida sigue siendo la de la fase 1, una caja 3D por examen. Se obtendrá combinando, en la fase 3, una predicción "¿este corte contiene el LCA?" (que da la extensión en z) con una caja 2D por corte (que da x e y).

Normalización de intensidad por volumen. Las intensidades de RM no tienen una escala física absoluta y varían entre exámenes. Siguiendo a nnU-Net para RM [10], cada volumen se recorta a sus propios percentiles 0.5 y 99.5 y luego se lleva a media 0 y desviación estándar 1 (Código 3). Como cada volumen se normaliza solo con sus propias estadísticas, la transformación **no aprende nada de otros volúmenes** y puede aplicarse a la partición de prueba sin fuga.

```python
def normalize_volume(volume, clip_low_pct, clip_high_pct):
    # Cast first: subtracting the mean from uint16 would wrap negatives around
    vol = volume.astype(np.float32)
    low, high = np.percentile(vol, [clip_low_pct, clip_high_pct])
    vol = np.clip(vol, low, high)
    mean, std = vol.mean(), vol.std()
    if std == 0:  # a constant volume has no contrast to scale
        return np.zeros_like(vol)
    return ((vol - mean) / std).astype(np.float32)
```

Código 3: Normalización por volumen (`src/data/transforms.py`).

Tamaño en plano uniforme. 914 de los 917 volúmenes miden 320×320 en plano; 6 tienen alguna dimensión menor (entre 290 y 316 píxeles). Se optó por **aplicar padding** hasta 320×320 en lugar de redimensionar, por tres razones: no se interpola ni se pierde detalle; el padding se coloca al final de cada eje (abajo y a la derecha), de modo que el píxel $(0, 0)$ no se mueve y las coordenadas de la caja **no requieren ningún ajuste**; y el valor de padding es el mínimo del volumen normalizado, que corresponde a fondo oscuro (tras el puntaje z, el valor 0 sería intensidad promedio, no fondo). El tamaño 320 es una constante del protocolo de adquisición y no una estadística aprendida, aunque se observó en el conjunto completo. El redimensionamiento al tamaño de entrada de la red (p. ej. 224×224) se pospone a la carga de datos en la fase 3, lo que permite probar distintos tamaños sin regenerar los datos.

Almacenamiento. Cada volumen procesado se guarda como un arreglo `.npy` en `float16`, nombrado por su clave (`<examId>-<seriesNo>.npy`). Con `float32` el conjunto procesado ocuparía alrededor de 11 GB; con `float16` ocupa 5.4 GB, menos que los datos crudos.

Decisiones explícitamente pospuestas. Reflejar las rodillas izquierdas para que todas tengan la misma orientación no se aplica en esta fase: implicaría también reflejar `roiX`, y es más flexible tratarlo como aumentación aleatoria en la fase 3. Por la misma razón, toda la aumentación (rotaciones pequeñas, cambios de contraste) se aplicará al vuelo y solo a los pliegues de entrenamiento.

2.7. Índice de cortes 2.5D

La función `build_slice_index` expande la tabla por volumen a **una fila por corte**, con las columnas `examId`, `seriesNo`, `slice_idx`, `split`, `aclDiagnosis`, `contains_acl`, `dist_to_roi`, la caja 2D (`x0`, `y0`, `x1`, `y1`) y la ruta del volumen procesado. Las definiciones son:

- Un corte es positivo si `roiZ` $\leq s <$ `roiZ`+`roiDepth`, con índices desde 0 y extremo superior excluido, la misma convención de la puerta de calidad.
- `dist_to_roi` es la distancia, en cortes, al corte positivo más cercano (0 dentro de la caja). Permite muestrear *negativos difíciles*, es decir, cortes vecinos al LCA.

- La caja 2D es $(x_0, y_0, x_1, y_1) = (\texttt{roiX}, \texttt{roiY}, \texttt{roiX} + \texttt{roiWidth}, \texttt{roiY} + \texttt{roiHeight})$, también con extremo excluido, y queda vacía en los cortes negativos. Como el conjunto anota una caja 3D, todos los cortes positivos de un volumen comparten la misma caja 2D.
- En la fase 3, el corte s se cargará junto con $s-1$ y $s+1$; en los bordes del volumen se repite el corte extremo.

La función incluye autoverificaciones (Código 4): el número de filas debe igualar la suma de profundidades; el número de positivos, la suma de `roiDepth`; y cada volumen debe tener exactamente `roiDepth` cortes positivos. La última comprobación detecta errores que se compensarían entre volúmenes en los totales.

```
slices = vols.loc[np.repeat(vols.index, depths)].reset_index(drop=True)
slices['slice_idx'] = np.concatenate([np.arange(d) for d in depths])
slices['contains_acl'] = ((slices['slice_idx'] >= slices['roiZ'])
                          & (slices['slice_idx'] < slices['roiZ'] + slices['roiDepth'
    ]))
above = slices['roiZ'] - slices['slice_idx']
below = slices['slice_idx'] - (slices['roiZ'] + slices['roiDepth'] - 1)
slices['dist_to_roi'] = np.maximum(np.maximum(above, below), 0)

assert len(slices) == vols['depth'].sum()
assert slices['contains_acl'].sum() == vols['roiDepth'].sum()
# ...and every volume has exactly roiDepth positive slices
```

Código 4: Expansión a cortes y autoverificaciones (`src/data/slices.py`, abreviado).

2.8. Estrategia para el desbalance

El problema tiene dos desbalances distintos, y cada uno requiere un tratamiento propio (Tabla 3). El muestreo y la ponderación se ejecutan al entrenar, en la fase 3; en esta fase se define y justifica la estrategia y se generan los datos que la hacen posible (`contains_acl`, `dist_to_roi` y `aclDiagnosis` por corte).
Se aplican tres reglas en todos los casos. Primero, el muestreo y los pesos se aplican **solo a los pliegues de entrenamiento**; validación y prueba conservan su distribución natural. Segundo, los pesos se **recalculan en cada pliegue** de validación cruzada, porque cada pliegue entrena con volúmenes distintos. Tercero, la exactitud no se usa como métrica: para la cabeza de clasificación se usará F1 o el área bajo la curva precisión-exhaustividad. Se descartó explícitamente SMOTE [14]: interpolar entre dos cortes de RM produce una imagen anatómicamente inválida, no una rodilla nueva.

2.9. Reproducibilidad

El pipeline completo se regenera con un solo comando desde la raíz del proyecto:

```
python -m scripts.build_dataset
```

La raíz del proyecto se calcula a partir de la ubicación del script (`Path(__file__)`) y no del directorio de trabajo, de modo que el resultado es idéntico sin importar desde dónde se

Cuadro 3: Estrategia de desbalance adoptada.

Desbalance	Técnica	Justificación
Cortes sin/con LCA (≈8.2:1)	Separar la tarea en dos cabezas: clasificación del corte y regresión de la caja, esta última entrenada solo con cortes positivos.	La regresión de la caja deja de tener desbalance; el desbalance queda aislado en una tarea binaria.
	Pérdida ponderada (`pos_weight` $= N_{neg}/N_{pos}$) en la cabeza de clasificación.	Sencilla, sin descartar datos; alternativa a evaluar: pérdida focal [13].
	Submuestreo de negativos resorteado en cada época, con preferencia por negativos difíciles (`dist_to_roi` ≤ 2).	Lotes balanceados sin pérdida permanente de información; los cortes vecinos son los que el modelo confunde.
Diagnóstico (75/19/6)	Sobremuestreo moderado de volúmenes por clase (pesos $\propto 1/\sqrt{\text{frecuencia}}$), combinado con aumentación.	La rotura es la clase más difícil de localizar; el peso inverso completo (12.5×) repetiría demasiado los mismos 46 volúmenes y favorecería el sobreajuste.
	Reportar el IoU por clase, no solo el global.	Con 6 % de roturas, el promedio global puede ocultar un fallo total en esa clase.

ejecute. Esto se verificó ejecutándolo desde dos directorios distintos. La semilla está en la configuración, y se comprobó que ejecuciones repetidas producen archivos idénticos byte a byte (Tabla 4).

Cuadro 4: Hashes SHA-1 de los artefactos tabulares; ejecuciones repetidas los reproducen exactamente.

Artefacto	SHA-1
`datasets/interim/manifest.csv`	`2d7604fb10138b4c3c940135e836303c08aa2984`
`datasets/interim/qa_report.csv`	`654e8df57f80dc55462cbb2ea65afb1c27ea6046`
`datasets/interim/splits.csv`	`0877f2eeebace2b8e069e4f91f961a93d9ec4d58`
`datasets/processed/slices.csv`	`d4636c0ec821739e54167283ddad06f254813d7a`

2.10. Pruebas automatizadas y documentación

Las verificaciones que el pipeline aplica sobre los datos reales se complementan con 31 pruebas unitarias (`pytest`) en `tests/`, que ejercitan cada función sobre datos sintéticos pequeños y, por tanto, se ejecutan en poco más de un segundo y no requieren descargar el conjunto (Tabla 5). Su propósito es distinto: las verificaciones del pipeline comprueban que *estos datos* están bien, y las pruebas comprueban que *el código* detecta los problemas cuando existen. Por eso varias pruebas introducen deliberadamente un error y exigen que

se reporte. Por ejemplo, cada una de las diez reglas de la puerta de calidad se prueba con un valor inválido. Un archivo `README.md` en la raíz del proyecto documenta la preparación del ambiente, la obtención de los datos crudos, el comando de reconstrucción y los hashes esperados de la Tabla 4.

Cuadro 5: Pruebas unitarias (31 en total; todas pasan).

Módulo	Qué se verifica
`manifest`	lectura de forma, tipo e intensidades; un archivo faltante se registra y no se descarta; una clave duplicada falla antes de abrir archivos
`qa`	un manifiesto válido produce un reporte vacío; cada regla (etiquetas, tamaño y límites de la caja en los tres ejes, tipo, intensidad, archivo presente) falla con un valor inválido
`split`	ningún examen en dos particiones; cada volumen asignado una vez; misma semilla, misma partición; la entrada no se modifica; un volumen faltante y una tolerancia demasiado estricta detienen el proceso
`transforms`	media 0 y desviación 1; sin desbordamiento de `uint16`; recorte de valores atípicos; volumen constante; el padding conserva los píxeles originales en su posición
`slices`	una fila por corte; cortes positivos, `dist_to_roi` y caja con extremo excluido verificados contra valores calculados a mano

3. Discusión de resultados

3.1. Integridad y calidad de los datos

Con la descarga completa, el manifiesto registra los 917 volúmenes y la puerta de calidad terminó con **cero problemas**. Lo que confirmó cada verificación:

- Todas las etiquetas son válidas y todas las cajas están dentro de su volumen.
- Todos los volúmenes son `uint16`, la intensidad mínima es 0 en todos y la máxima no supera 4095, lo que confirma con los datos la afirmación del artículo original de que las imágenes son de 12 bits [1].
- Hay 25 formas distintas de volumen. La variación está casi toda en la profundidad (21 a 45 cortes, media 30.9, DE 2.3); en el plano, solo 6 volúmenes no miden 320×320, por lo que el padding afecta a menos del 1 % de los datos.
- La profundidad de la caja es de 3 cortes en 505 volúmenes y de 4 en 291; los casos extremos son 71 volúmenes con 2 cortes y 10 con 6.

Variabilidad de intensidad. El máximo de intensidad por volumen tiene media 2438 (DE 353), pero 10 volúmenes alcanzan el tope de 4095 (saturación) y 4 no superan 1000. La Figura 2 muestra el volumen más tenue del conjunto (máximo 563) junto a uno típico:

en crudo sus distribuciones apenas se traslapan, y tras la normalización por volumen ambas quedan en una escala comparable. Una normalización con estadísticas globales habría dejado el volumen tenue comprimido cerca de cero, y el recorte por percentiles evita que los vóxeles saturados dominen la escala. Esto respalda empíricamente la decisión de normalizar por volumen. Sobre una muestra de 20 volúmenes procesados se verificó media 0.000 y desviación 1.000 en la región sin padding, y un error máximo de $1{,}95 \times 10^{-3}$ por almacenar en `float16`, despreciable frente a una escala de desviación unitaria.

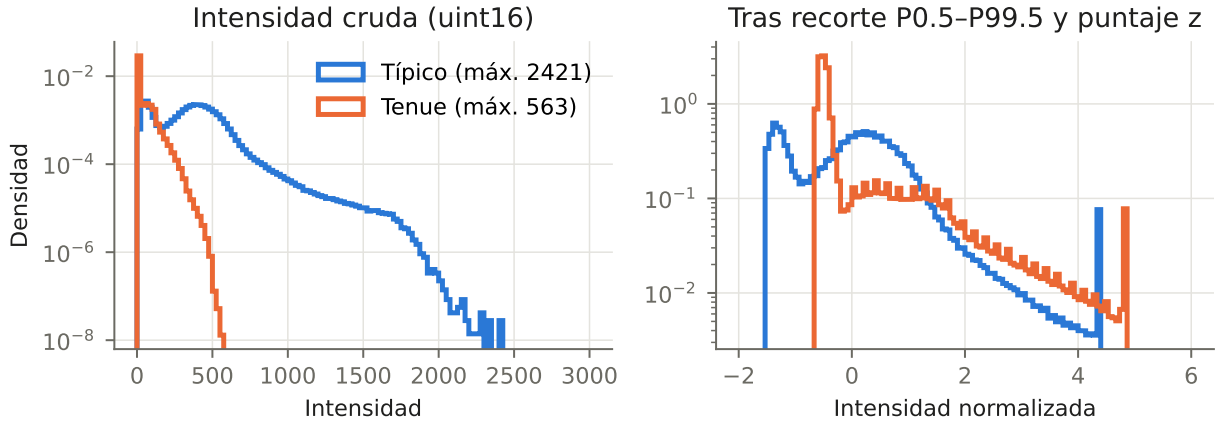


Figura 2: Histogramas (escala logarítmica) de un volumen típico y del volumen más tenue del conjunto, antes y después de la normalización por volumen. Las barras verticales a la derecha del panel normalizado son los vóxeles acumulados en el percentil 99.5 por el recorte.

Inspección visual de las cajas. La Figura 3 superpone la caja de la anotación sobre un corte positivo y un negativo difícil de un volumen de cada diagnóstico, ya procesado. Confirma tres cosas: `roiX` corresponde a la columna y `roiY` a la fila (los ejes no están intercambiados), el padding inferior no desplaza las cajas, y la caja abarca la escotadura intercondílea donde se ubica el LCA. También ilustra la limitación señalada en la fase 1: la caja es amplia e incluye parte de los cóndilos, en lugar de ajustarse al ligamento. Los negativos difíciles (a dos cortes de distancia) son visualmente muy parecidos a los positivos, lo que justifica muestrearlos preferentemente.

Para cumplir el criterio de la fase 1 (revisar al menos 30 ejemplos), se inspeccionó además una muestra aleatoria de 30 volúmenes, 10 por diagnóstico, escogida con semilla fija, mostrando el corte positivo central de cada uno (Figura 4). En los 30 casos la caja se ubica sobre la escotadura intercondílea, entre la cara posterior del cóndilo femoral y la meseta tibial, y no se encontró ninguna caja desplazada fuera de la articulación ni con los ejes intercambiados. En todos los casos la caja es sistemáticamente más amplia que el ligamento: incluye parte del cóndilo femoral y de la meseta tibial. Esto confirma, sobre una muestra mayor, que el objetivo es una región de interés amplia y no una caja ajustada. Esta inspección no es una validación clínica, porque no la hizo un especialista, y así debe interpretarse.

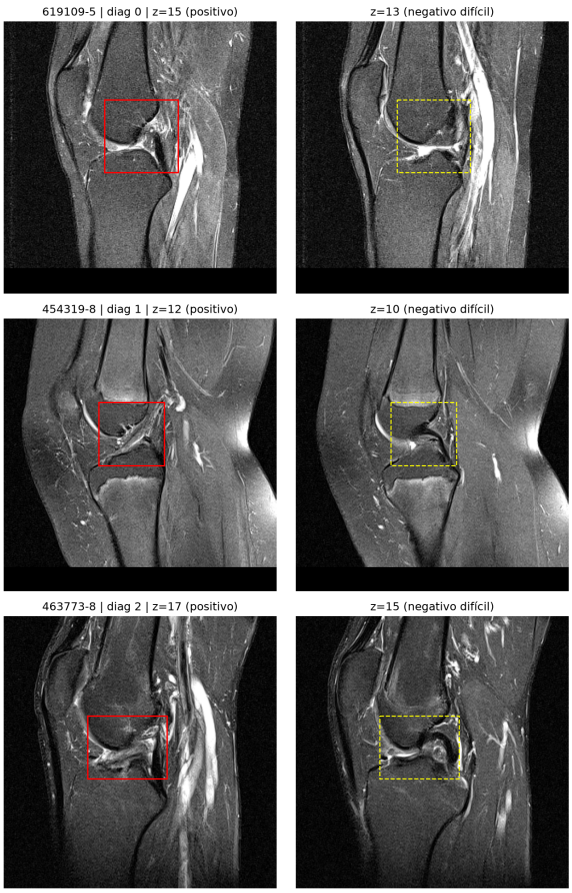


Figura 3: Caja anotada sobre volúmenes procesados, uno por diagnóstico (0 sano, 1 parcial, 2 roto). Izquierda: corte positivo. Derecha: negativo difícil a dos cortes de la caja. La banda negra inferior en las dos primeras filas es el padding hasta 320×320.

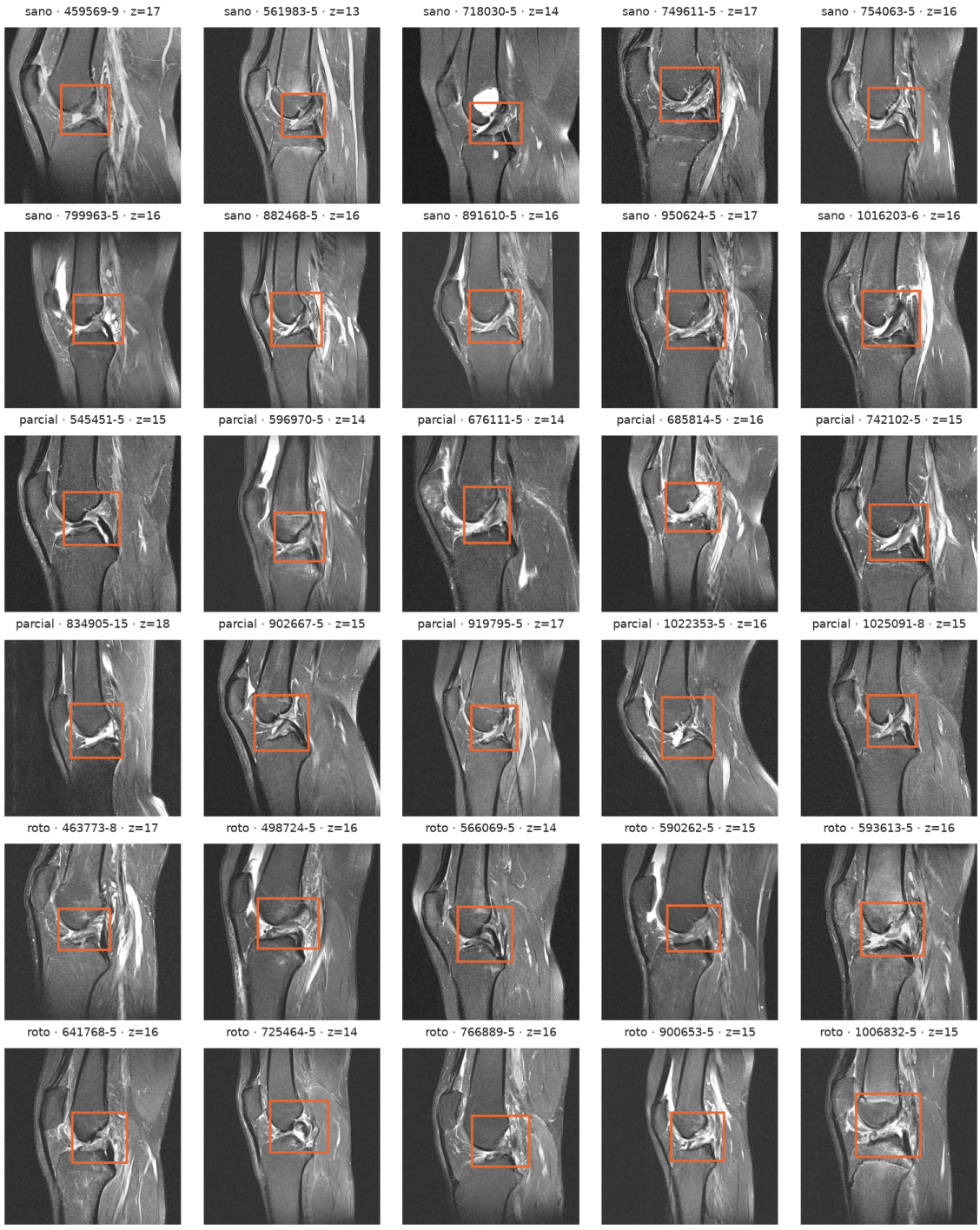


Figura 4: Inspección visual de 30 volúmenes procesados (10 por diagnóstico, muestra aleatoria con semilla 0): corte positivo central de cada volumen con su caja anotada.

3.2. Partición

La Tabla 6 muestra la partición resultante. Todas las particiones tienen entre 152 y 153 volúmenes, y la mayor desviación de proporción respecto del total es de **0.54 puntos porcentuales** (en `fold_3`, que tiene 10 roturas en lugar de 9). Ninguna partición se aleja del ideal en más de un paciente por clase, lo cual es prácticamente el óptimo alcanzable con grupos y solo 55 casos de rotura. Los 7 exámenes con varias series quedaron cada uno completo en una sola partición.

Cuadro 6: Volúmenes por partición y diagnóstico.

Partición	Sano	Parcial	Roto	Total
`fold_0`	115	29	9	153
`fold_1`	115	29	9	153
`fold_2`	115	29	9	153
`fold_3`	115	28	10	153
`fold_4`	115	28	9	152
`test`	115	29	9	153
Total	690	172	55	917

3.3. Índice de cortes y desbalance

El índice contiene **28 359 cortes**, de los cuales 3 081 (10.86 %) contienen el LCA (Tabla 7). La proporción de positivos es muy estable entre particiones (10.67 % a 11.15 %), y cada partición tiene entre 29 y 34 cortes positivos de volúmenes con rotura. El cociente de negativos a positivos en el entrenamiento de cada pliegue varía apenas entre 8.22 y 8.29, de modo que el `pos_weight` será prácticamente el mismo en toda la validación cruzada, aunque se recalcule en cada pliegue como se estableció.

Cuadro 7: Cortes por partición. "Roto (+)" cuenta los cortes positivos de volúmenes con rotura completa.

Partición	Negativos	Positivos	Total	% positivos	Roto (+)
`fold_0`	4 230	505	4 735	10.67	29
`fold_1`	4 225	514	4 739	10.85	30
`fold_2`	4 199	519	4 718	11.00	32
`fold_3`	4 228	513	4 741	10.82	34
`fold_4`	4 229	507	4 736	10.71	30
`test`	4 167	523	4 690	11.15	31
Total	25 278	3 081	28 359	10.86	186

Dos observaciones adicionales informan la fase 3:

- **La caja abarca en promedio 3.4 cortes en las tres clases** (2 315/690, 580/172 y 186/55 dan 3.36–3.38). La rotura no cambia la extensión de la anotación en z; lo que

cambia es la apariencia del ligamento dentro de ella. Por eso el reto de la clase minoritaria es de apariencia y no de geometría.

- **Hay 3 668 negativos difíciles** (`dist_to_roi` ≤ 2), el 14.5 % de los negativos. Todos los volúmenes tienen al menos seis cortes de margen a cada lado de la caja, así que siempre existen negativos difíciles y fáciles para muestrear.

En el conjunto de desarrollo (764 volúmenes: 575 sanos, 143 parciales, 46 rotos), los pesos de sobremuestreo moderado resultan 1 : 2.0 : 3.5, frente a 1 : 4.0 : 12.5 con peso inverso completo. Esto cuantifica por qué se prefirió la versión moderada.

3.4. Seguimiento de la tabla de riesgos de la fase 1

La Tabla 8 contrasta los riesgos de la fase 1 que correspondían a esta fase con su criterio verificable y el resultado obtenido.

Cuadro 8: Estado de los riesgos de la fase 1 al cierre de la fase 2.

Riesgo (fase 1)	Criterio Q	Resultado
19.7 % de volúmenes faltantes	0 registros usados sin archivo verificado	**Resuelto**: descarga completa desde Zenodo, 917/917 verificados; la verificación quedó automatizada en la puerta de calidad.
Desbalance sesga los splits	Proporciones por partición a ±2 p.p. del total	**Cumplido**: desviación máxima de 0.54 p.p., verificada en código.
Fuga entre splits	Intersección vacía de `examId`	**Cumplido**: verificado en código; el pipeline se detiene si falla.
Objeto pequeño	IoU de validación $\geq$ línea base	**Mitigado en diseño**: se adoptó la contingencia prevista (representación 2.5D); el criterio se medirá en la fase 3.
Caja amplia, no ajustada	Revisar visualmente $\geq$30 ejemplos	**Cumplido**: 30 volúmenes inspeccionados (10 por clase); todas las cajas sobre la escotadura intercondílea y sistemáticamente más amplias que el ligamento (limitación confirmada, no corregible).

3.5. Limitaciones

- **Sin identificador de paciente.** La agrupación por `examId` es la mejor disponible, pero una misma persona podría tener dos exámenes distintos (ambas rodillas o un control posterior). Esta fuente residual de fuga no puede verificarse con los datos.
- **Pocas roturas en prueba.** Con 9 volúmenes con rotura en prueba, cada uno pesa alrededor del 11 % del IoU de esa clase; su estimación tendrá una incertidumbre alta y así deberá reportarse.

- **Caja amplia.** La anotación no es una caja mínima alrededor del ligamento; el IoU medirá el acuerdo con esa anotación y no con la anatomía exacta.
- **Un único centro y equipo.** Nada en esta fase mitiga el sesgo de dominio identificado en la fase 1.
- **Licencia.** KneeMRI se distribuye con licencia CC BY-NC-ND 4.0, que permite el uso académico no comercial; los datos procesados no se redistribuyen.

4. Avance y revisión del plan

El plan de la fase 1 situaba la preparación de datos entre las semanas 5 y 9, con cierre el **6 de octubre**, según la carta al estudiante. Posteriormente, la fecha de entrega de este documento se trasladó al 16 de octubre, así que el entregable se presenta **a tiempo respecto al calendario vigente del curso**. Medido contra el plan propio, sin embargo, la situación es otra: respecto al plan de la fase 1, **el proyecto va atrasado aproximadamente diez días**, y el cambio de fecha del curso absorbió ese atraso sin eliminarlo. Las causas principales fueron:

1. El descubrimiento de que faltaba el 20 % de los volúmenes, que obligó a obtener y verificar una segunda fuente antes de poder particionar.
2. Varias iteraciones de arquitectura (rutas dependientes del directorio de trabajo, funciones que leían configuración o reconstruían el manifiesto), que se corrigieron antes de avanzar porque afectaban directamente la reproducibilidad.
3. La decisión entre modelado clásico y aprendizaje profundo, de la cual dependían las transformaciones, que se tomó a mitad de la fase.

La Tabla 9 resume el estado de lo comprometido para esta fase. Todo quedó completo, incluidos tres elementos (pruebas, README e inspección de 30 cajas) que se cerraron el mismo día de la entrega.

Consecuencias para el resto del plan. Si se mantiene el 27 de octubre como fecha de las fases 3 (modelado) y 4 (evaluación), quedan once días para instalar PyTorch, implementar el cargador 2.5D con la estrategia de desbalance, entrenar y evaluar, en lugar de las tres semanas que preveía el plan. Es un plazo ajustado. Para absorber el atraso sin sacrificar la calidad se propone:

- Comenzar con el modelo más simple que cumple el planteamiento: un único *backbone* preentrenado con las dos cabezas descritas, evaluado primero contra la línea base (IoU 0.391) antes de cualquier optimización de hiperparámetros.
- Reducir, si es necesario, la búsqueda de hiperparámetros de la fase 3 a pocos ensayos, sin reducir la validación cruzada ni la evaluación por clase.

Cuadro 9: Estado de los entregables de la fase 2 comprometidos en el plan.

Compromiso	Estado
Verificar la disponibilidad real de archivos	Completo (917/917, automatizado)
Splits estratificados y agrupados por examen	Completo y verificado (`splits.csv`)
Normalizar volúmenes de profundidad variable	Completo, con cambio justificado: no se remuestrea la profundidad (representación 2.5D)
Decidir la representación (3D frente a 2.5D)	Completo: 2.5D
Pipeline documentado y reproducible	Completo: un comando, configuración en un solo archivo, hashes verificados
Pruebas automatizadas (`pytest`)	Completo: 31 pruebas, todas pasan
README con instrucciones de reconstrucción	Completo (`README.md`)
Inspección visual de $\geq$30 cajas	Completo (30 de 30, Figura 4)

- Reutilizar directamente lo construido en esta fase: `splits.csv` define los pliegues, `slices.csv` contiene todo lo necesario para el muestreo, y las pruebas unitarias protegen el código de datos frente a cambios durante el modelado.

El proceso es iterativo: si la fase 3 muestra que la representación 2.5D no supera la línea base, se volverá a esta fase para reconsiderar la representación, por ejemplo con un recorte alrededor de la región típica del LCA. El pipeline permite hacerlo cambiando la configuración y volviendo a ejecutar un comando.

5. Conclusiones

Esta fase convirtió el conjunto KneeMRI en datos listos para entrenar un modelo de localización del LCA mediante un pipeline automatizado, verificable y reproducible. La pregunta que guió cada decisión fue cómo saber que la tarea quedó bien hecha, y la respuesta fue convertir los criterios de calidad de la fase 1 en código que detiene el proceso cuando no se cumplen.

Los resultados más importantes son tres. Primero, el problema de datos faltantes heredado de la fase 1 quedó resuelto y verificado: el proyecto trabaja con los 917 volúmenes, y la puerta de calidad confirmó con los datos que las etiquetas, las cajas y las intensidades son consistentes. Segundo, el riesgo de fuga, el más grave de esta fase, quedó controlado por construcción: la partición agrupada y estratificada se hace antes de cualquier transformación, se congela en un archivo y su verificación es parte del pipeline; además, todas las transformaciones son por volumen y no aprenden de otros datos. Tercero, la elección de una representación 2.5D, junto con el índice de cortes, deja preparada una estrategia concreta y cuantificada para los dos desbalances del problema, que se aplicará en la fase 3 solo sobre los pliegues de entrenamiento.

Persisten limitaciones que no dependen de esta fase: la ausencia de un identificador de paciente, la amplitud de la caja anotada, el reducido número de roturas en prueba y el

origen de un único centro. Todas quedan documentadas para matizar la interpretación de los resultados futuros. Finalmente, el entregable llega a tiempo según el calendario vigente del curso, pero el proyecto va unos diez días atrás respecto a su propio plan, y ese atraso reduce el tiempo disponible para el modelado. El atraso es real, pero la base construida en esta fase reduce el riesgo de las siguientes. El pipeline está probado, documentado y verificado, de modo que el modelado podrá concentrarse en el modelo, porque los datos ya no son una fuente de incertidumbre.

Referencias

[1] I. Štajduhar, M. Mamula, D. Miletić, and G. Ünal, "Semi-automated detection of anterior cruciate ligament injury from MRI," *Computer Methods and Programs in Biomedicine*, vol. 140, pp. 151–164, 2017.

[2] I. Štajduhar, M. Mamula, D. Miletić, and G. Ünal, "KneeMRI dataset." Zenodo, registro 14789903, 2017. Versión v1, licencia CC BY-NC-ND 4.0. https://zenodo.org/records/14789903.

[3] S. Studer, T. B. Bui, C. Drescher, A. Hanuschkin, L. Winkler, S. Peters, and K.-R. Müller, "Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology," *Machine Learning and Knowledge Extraction*, vol. 3, no. 2, pp. 392–413, 2021.

[4] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[5] S. Anwaar, "Knee MRI Dataset." Kaggle, 2020. Derivado del conjunto de datos original de Štajduhar et al.

[6] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[7] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010.

[8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] DrivenData, "Cookiecutter data science." https://cookiecutter-data-science.drivendata.org/, 2024. Consultado en octubre de 2026.

[10] F. Isensee, P. F. Jaeger, S. A. A. Kohl, J. Petersen, and K. H. Maier-Hein, "nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation," *Nature Methods*, vol. 18, pp. 203–211, 2021.

[11] N. Bien, P. Rajpurkar, R. L. Ball, J. Irvin, A. Park, E. Jones, M. Bereket, B. N. Patel, K. W. Yeom, K. Shpanskaya, S. Halabi, E. Zucker, G. Fanton, D. F. Amanatullah, C. F. Beaulieu, G. M. Riley, R. J. Stewart, F. G. Blankenberg, D. B. Larson, R. H. Jones, C. P. Langlotz, A. Y. Ng, and M. P. Lungren, "Deep-learning-assisted diagnosis for knee magnetic resonance imaging: Development and retrospective validation of MRNet," *PLoS Medicine*, vol. 15, no. 11, p. e1002699, 2018.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, 2017.

[14] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.